DetectPoints 機能仕様書

1. 概要

DetectPointsは、ハイキングマップのPNG画像から円形のポイントマーカーを自動検出し、座標とIDをJSON形式で出力するコンピュータビジョンシステムです。地図上の円形マーカー（臙脂色、赤色、白抜き）を認識し、近くにあるIDラベル（F4、F13、J11等）を抽出します。

2. システム構成

2.1 主要コンポーネント

- **detect_points.py**: ポイント検出・抽出メインスクリプト
- **verify_points.py**: 検出結果の確認・検証スクリプト
- **requirements.txt**: 依存関係定義ファイル
- **samples/**: サンプル画像とJSON出力データ

2.2 依存関係

- opencv-python (>= 4.8.0): 画像処理・コンピュータビジョン
- numpy (>= 1.24.0): 数値計算
- pytesseract (>= 0.3.10): OCR（光学文字認識）
- Pillow (>= 10.0.0): 画像ファイル処理

2.3 システム要件

- Tesseract OCRエンジンのシステムインストール
- 英語言語パック（ID認識用）
- Python 3.7以上

3. detect_points.py 機能仕様

3.1 目的

ハイキングマップのPNG画像から円形ポイントマーカー（臙脂、赤、白抜き）を検出し、ポイントIDを抽出してJSON出力する。入力ファイルと同じフォルダにJSON結果を出力。

3.2 主要クラス: HikingPointDetector

3.2.1 初期化

```
HikingPointDetector(image_path: str)
```

- 画像ファイルを読み込み、基本情報を取得
- 画像が存在しない場合はValueErrorを発生
- オリジナル画像のコピーを保持

3.2.2 円形検出 - detect_circles()

改良された検出アルゴリズム：

- **複数パラメータセット方式**: 2つの異なるパラメータセットで検出を実行
- **前処理**: ガウシアンブラー（9x9カーネル、σ=2）によるノイズ除去
- **パラメータセット1（標準検出）**:
  - 最小距離: 20ピクセル
  - Cannyエッジ上閾値: 50
  - 円検出閾値: 30
  - 半径範囲: 5-18ピクセル
- **パラメータセット2（高感度検出）**:
  - 最小距離: 18ピクセル
  - Cannyエッジ上閾値: 30
  - 円検出閾値: 20
  - 半径範囲: 4-15ピクセル
- **重複除去**: 距離ベースで重複する円を除去
- **出力**: 座標(x, y)と半径rのタプルリスト

3.2.3 色フィルタリング - filter_circles_by_color()

改良された色認識：

- **対象色範囲**:
  - 赤色系（臙脂・赤）: HSV範囲 [0-15, 165-180] H, [80-255] S, [50-255] V
  - 白色系（白抜き）: HSV範囲 [0-180] H, [0-30] S, [200-255] V
- **判定基準**（調整済み）:
  - 赤色ピクセル比率 > 6%
  - 白色ピクセル比率 > 15%
- **処理**: 円周辺のROI（関心領域）をHSV色空間で解析

3.2.4 テキスト抽出 - extract_text_near_point()

ID認識特化型OCR：

- **検索範囲**: 円半径の3倍の正方形領域
- **OCR設定**:
  - 言語: 英語のみ ('eng') - ID認識に特化
  - PSM設定: --psm 8（単語認識）
  - 文字制限: 英数字のみ（A-Z, 0-9）
- **前処理**: CLAHE（制限適応ヒストグラム平坦化）によるコントラスト強化
- **IDパターン抽出**: 正規表現 `[A-Z]{1,2}\d+` でIDパターンを検索
- **フォールバック**: OCR失敗時は座標ベースの名前を生成（point_x_y形式）

3.2.5 メイン処理 - detect_and_extract_points()

1. 複数パラメータでの円形ポイント検出
2. 重複除去処理

3. 色による絞り込み
4. 各ポイントのID抽出
5. JSON構造体の生成（絶対座標と相対座標の両方）

3.3 出力データ形式

3.3.1 JSON構造

```
{
  "image_info": {
    "filename": "画像ファイル名",
    "width": 画像幅（ピクセル）,
    "height": 画像高さ（ピクセル）
  },
  "points": [
    {
      "id": "ポイントID（F4、J11等）",
      "name": "ポイント名（IDと同じ）",
      "x": 絶対X座標,
      "y": 絶対Y座標,
      "relative_x": 相対X座標(0.0-1.0),
      "relative_y": 相対Y座標(0.0-1.0),
      "radius": 検出半径
    }
  ],
  "total_points": ポイント総数
}
```

3.3.2 座標系

- **絶対座標**: 画像左上を原点(0,0)とするピクセル座標
- **相対座標**: 画像サイズで正規化した0.0-1.0範囲の座標（4桁小数点精度）

3.4 出力ファイル

- **ファイル名**: `{入力ファイル名}_points.json`
- **出力先**: 入力PNGファイルと同じフォルダ
- **エンコーディング**: UTF-8
- **フォーマット**: インデント付きJSON（可読性向上）

3.5 コマンドライン引数

```
python detect_points.py input_image.png [-o output.json]
```

- `input_image`: 入力PNG画像ファイル（必須）
- `-o, --output`: 出力JSONファイル名（省略時は自動生成）

4. verify_points.py 機能仕様

4.1 目的

detect_points.pyの検出結果を視覚的に確認し、検証レポートを生成する。

4.2 主要機能

- 検出ポイントの可視化
- インタラクティブな確認機能
- 統計レポート生成

5. 技術仕様

5.1 画像処理アルゴリズム

- **円検出**: OpenCV HoughCircles（勾配法）の複数パラメータ実行
- **色空間**: BGR→HSV変換による色範囲指定
- **ノイズ除去**: ガウシアンブラー（9x9、σ=2）
- **コントラスト強化**: CLAHE適用（clipLimit=2.0、tileGridSize=8x8）
- **重複除去**: ユークリッド距離による重複判定（閾値: (r1+r2)*0.7）

5.2 OCR処理

- **エンジン**: Tesseract OCR
- **特化設定**: 英数字のみの文字認識
- **パターンマッチング**: 正規表現によるIDパターン抽出
- **エラーハンドリング**: 失敗時の自動フォールバック

5.3 パフォーマンス特性

- **処理対象**: 10-200ポイント/画像（標準的には15前後）
- **検出精度**: 複数パラメータとフィルタリングによる高精度化
- **処理時間**: 画像サイズとポイント数に比例

5.4 品質保証

- **テストデータ**: samples/map-sample01.pngで15ポイント検出確認済み
- **ID認識実績**: F4、F13、J11、SG、OX、ER、CS等の形式に対応
- **座標精度**: 相対座標で4桁小数点精度を保証

6. エラーハンドリング

6.1 想定エラーと対応

- **画像読み込み失敗**: ValueErrorで詳細メッセージ出力
- **OCR処理エラー**: フォールバック名での継続処理
- **メモリ不足**: 適切なエラーメッセージで処理中断
- **出力ディレクトリアクセス失敗**: パーミッションエラーの詳細表示

6.2 ログ出力

- 検出プロセスの進捗表示
- 各段階での検出数報告
- エラー発生時の詳細情報

7. 制限事項

7.1 技術的制限

- **対象形状**: 円形ポイントのみ（楕円や多角形は非対応）
- **色範囲**: 赤色系と白色系のマーカーのみ
- **OCR依存**: Tesseractの認識精度に依存
- **ID形式**: アルファベット1-2文字+数字の形式のみ

7.2 運用制限

- **画像形式**: PNG形式のみ対応
- **言語パック**: Tesseract英語パックが必須
- **処理時間**: 高解像度画像や多数ポイントで処理時間増加
- **メモリ使用量**: 大きな画像での高メモリ消費

8. 今後の拡張可能性

8.1 機能拡張

- 他の図形（四角形、三角形）への対応
- カスタム色範囲設定機能
- 一括処理機能（複数画像対応）
- WebAPI化

8.2 精度改善

- 機械学習による形状認識
- 深層学習OCRエンジンの導入
- ユーザー補正機能の追加